Configuración del ADC y la UART del PIC16F18426 en el instrumento de corrientes de Eddy

Proyecto Micro 3 — Microcontroladores

Resumen de decisiones de diseño

Resumen

El PIC16F18426 digitaliza las componentes en fase (I) y en cuadratura (Q) que entrega la placa de demodulación y las envía a un PC por UART. Este documento reúne, en un solo lugar, *qué* se escogió para el ADC y la UART y *por qué*, con los cálculos mínimos de cada valor. Los datos viajan como texto ASCII, una línea `2048,2051\r\n` por par. Resultado: muestreo de I y Q a 1 kHz disparado por hardware, enlace a 115 200 baudios 8N1 y una línea cada dos pares (500 líneas/s), que ocupa el 48 % de la línea; una línea por par la dejaría al 96 %, sin margen.

I. Datos del sistema

PIC16F18426 (14 pines, 8 bits) con oscilador interno HFINTOSC a $F_{\text{osc}} = 16\,\text{MHz}$ ($\pm 2\,\%$), así que $T_{\text{CY}} = 4/F_{\text{osc}} = 250\,\text{ns}$. Alimentación $V_{DD} = 3{,}7\,\text{V}$, accesible en J1, en TP3 y en el pin 2 de J4. Las señales I (RC0) y Q (RC1) llegan centradas en $V_{\text{ref2}} = 1{,}85\,\text{V}$, con banda de $\lesssim 20\,\text{Hz}$. El enlace al PC es EUSART1 $\rightarrow$ traductores SN74LVC1T45 $\rightarrow$ CP2102 (USB) $\rightarrow$ LabVIEW o Python.

II. UART: qué se escogió y por qué

La Tabla I resume cada decisión; abajo están los cálculos.

A. Cálculo del baud

Con `BRGH` = 1 y `BRG16` = 1:

$$\text{Baud} = \frac{F_{\text{osc}}}{4\,(\text{SP1BRG} + 1)} = \frac{16\,\text{MHz}}{4\,(34 + 1)} = 114\,286\ \text{baud} \tag{1}$$

$$\text{error} = \frac{114\,286 - 115\,200}{115\,200} = -0{,}79\,\% \tag{2}$$

La fórmula de 8 bits, $\text{Baud} = F_{\text{osc}}/[16(\text{SP1BRG} + 1)]$, da $\text{SP1BRG} = 7{,}68 \rightarrow 8$ y 111 111 baud ($-3{,}55\,\%$), demasiado lejos si se suma el $\pm 2\,\%$ del oscilador.

Tolerancia. El receptor muestrea el bit de parada a 9,5 bits del flanco de inicio, así que el desfase total debe ser menor que $0{,}5/9{,}5 = \pm 5{,}3\,\%$ (en la práctica, entre 2 % y 3 %). Peor caso: $-0{,}79\,\% - 2\,\% \approx -2{,}8\,\%$: funciona.

Tiempos. $T_{\text{bit}} = 8{,}75\,\mu\text{s}$, byte 8N1 (10 bits) $= 87{,}5\,\mu\text{s}$, capacidad $\approx 11\,429\,\text{B/s}$.

Verificado: eco a 115 200 baud con 20/20, 256/256 y 1000/1000 bytes sin errores (2026-10-19).

III. ADC: qué se escogió y por qué

La Tabla II resume cada decisión; abajo están los cálculos.

TABLA I
DECISIONES DE LA UART

Parámetro	Elegido	Por qué
Modo	EUSART1 asíncrono, **8N1**	Estándar: lo acepta cualquier terminal, LabVIEW o Python sin configuración especial.
Velocidad	115 200 **baud** (114 286 reales)	Es el baud estándar más bajo donde caben 500 líneas/s de 11 bytes: ocupan el 48 %. A 9 600 y 19 200 baud serían el 573 % y el 286 %; a 57 600, el 95 %, sin margen.
Generador de baud	`BRGH` = 1, `BRG16` = 1, `SP1BRG` = 34	Error de $-0{,}79$ %; con el generador de 8 bits sería $-3{,}55$ %.
Pines	TX = RC4, RX = RC5 (PPS)	No chocan con el ADC (RC0, RC1) ni con el programador (RA0, RA1, RA3).
Nivel de RX	TTL (`INLVLC5` = 0)	$V_{IH} = 1{,}73$ V: 1,57 V de margen con 3,3 V. Con Schmitt serían 2,96 V.
Capa física	2 × SN74LVC1T45	El CP2102 admite como máximo $V_{IO} + 0{,}5$ V $= 3{,}8$ V; el PIC saca 3,7 V.
Atención	Interrupción de TX	Nada espera: prohibidos `while(!TX1IF)` y `__delay_ms`. El RX queda configurado, pero el PIC no recibe comandos.
Buffers	Circulares: RX 32 B, TX 64 B	Potencias de 2: el índice avanza con `(i+1) & (N-1)`, sin división. TX guarda 4 líneas.
Envío	`tx_DAT()`, `tx_string()`	Todo o nada: nunca sale media línea; si no cabe, devuelven `false` sin esperar.
Formato	**Texto ASCII**, `xxxx,xxxx\r\n`	Ver la sección IV.

A. *Cálculos*

$$1\ \text{LSB} = \frac{V_{DD}}{4096} = 0{,}903\,3\ \text{mV}, \qquad N(1{,}85\ \text{V}) = \frac{1{,}85}{3{,}7} \times 4096 = 2048 \tag{3}$$

$$T_s = (\texttt{TMR0H} + 1) \cdot \frac{16}{F_{\text{osc}}/4} = 250 \times 4\,\mu\text{s} = 1\,000\,\mu\text{s} \tag{4}$$

$$T_{\text{conv}} = 14\,T_{AD} + 2\,T_{\text{CY}} = 14(2\,\mu\text{s}) + 2(0{,}25\,\mu\text{s}) = 28{,}5\,\mu\text{s} \tag{5}$$

Calidad. El ideal de 12 bits es SINAD $= 6{,}02 \cdot 12 + 1{,}76 = 74$ dB. El requisito de 60 dB equivale a ENOB $= (60 - 1{,}76)/6{,}02 = 9{,}67$ bits: con una senoidal a escala completa (1,31 V rms) se permiten 1,31 mV rms de ruido y distorsión, unos 1,45 LSB. El ruido de cuantización es LSB$/\sqrt{12} = 0{,}261$ mV rms.

Sesgo entre I y Q. La muestra de Q se toma $\approx 46\,\mu$s después de la de I (con 10 µs de adquisición). A 20 Hz eso es $360^\circ \cdot 20 \cdot 46\,\mu\text{s} = 0{,}33^\circ$ de error de fase: despreciable.

Red de 1 kΩ + 10 nF. Al conectarse el capacitor interno ($C_{\text{HOLD}} = 28$ pF), el pin salta solo $28/10\,028 = 0{,}28$ % de la diferencia con el canal anterior. Luego se recupera con $\tau = 1\,\text{k}\Omega \cdot 10\,\text{nF} = 10\,\mu$s. La mezcla de I en Q queda en -60 dB con 10 µs de adquisición y en -86 dB con 40 µs; por eso no conviene acortarla por debajo de unos 40 µs.

TABLA II
DECISIONES DEL ADC

Parámetro	Elegido	Por qué
Convertidor	ADCC de 12 bits, modo básico, resultado a la derecha	4096 niveles; 1 LSB $= 0{,}903$ mV.
Referencia	$V_{\text{ref+}} = V_{DD} = 3{,}7$ V (ADREF = 0x00)	Cubre 0 a 3,7 V sin recorte y es ratiométrica: V_{ref2} sale de un divisor del mismo riel, así que el reposo queda en 2048 aunque V_{DD} varíe. FVR de $2{,}048$ V: recorta; FVR de $4{,}096$ V: no puede superar V_{DD}.
Canales	I = RC0 (0x10), Q = RC1 (0x11)	Un solo ADC multiplexado: I y luego Q en ráfaga.
Muestreo	$f_s = 1$ kHz	Pedido por el profesor. Con señal $\lesssim 20$ Hz: sobremuestreo de 25; Nyquist en 500 Hz. Al PC llega 1 de cada 2 pares (500 Hz).
Disparo	Timer0 por hardware (ADACT = 0x02)	El instante de I no depende del software (sin jitter de código). El Timer2 queda para el PWM.
Timer0	$F_{\text{osc}}/4$, prescaler 1:16, TMR0H = 249	Periodo exacto de $1\,000$ µs sin recargar por software.
Reloj del ADC	$T_{AD} = 2$ µs (ADCLK $= F_{\text{osc}}/32$)	Dentro del rango permitido (0,5 a 9 µs); conversión de $28{,}5$ µs.
Adquisición	ADACQ = 160	El datasheet es ambiguo: 10, 40 o 320 µs. Los tres casos caben en 1 ms; se mide con el pin marcador.
Promedio	Ninguno	Cada línea lleva una sola muestra de I y una de Q.
Entrada al pin	1 kΩ en serie + 10 nF a GND	Aísla al operacional MCP604 de la carga del S&H y sirve de depósito de carga. Pide $T_{\text{ACQ}} \gtrsim 40$ µs (ver abajo).

IV. FORMATO DE LOS DATOS: TEXTO ASCII

A. *La línea que se transmite*

TABLA III
LÍNEA DE TEXTO. EJEMPLO: $I = 2048$, $Q = 2051$

Byte	0	1	2	3	4	5	6	7	8	9	10
Carácter	2	0	4	8	,	2	0	5	1	CR	LF
Hex	32	30	34	38	2C	32	30	35	31	0D	0A

I y Q van en decimal, de 0 a 4095, sin ceros a la izquierda: hasta 11 bytes por línea. Primero va I y después Q, sin nombres: el orden dice cuál es cuál. Dos números separados por coma los dibuja directo el Serial Plotter de Arduino. El PC lee hasta `\n` y separa los números con `%d,%d`; una línea con otra forma se descarta.

B. *Cuántas líneas por segundo*

Una línea de 11 bytes tarda $11 \times 87{,}5$ µs $= 962{,}5$ µs, casi todo el periodo de muestreo. Por eso el PIC manda una línea cada dos pares (ADC_PARES_POR_LINEA = 2), como resume la Tabla IV.

TABLA IV
OCUPACIÓN DE LA LÍNEA A 114 286 BAUD

	1 línea por par	**1 cada 2 pares (elegido)**	**1 cada 50 pares**
Líneas por segundo (f_s en el PC)	1000	**500**	20
Bytes por segundo	11 000	5 500	220
Ocupación	96 %: **sin margen**	48 %	1,9 %
Nyquist en el PC	500 Hz	250 Hz	10 Hz
Sirve para SINAD a 5 y 20 Hz	—	**sí** (armónicos 2 a 5 $\leq$ 100 Hz)	no

C. *Conversión a decimal en el PIC*

El PIC16 no tiene divisor por hardware, así que `a_decimal()` saca cada cifra restando 1000, 100, 10 y 1 tantas veces como quepa (hasta 18 restas de 16 bits para 4095) y le suma `'0'` (`0x30`). Previsión: ≈ 95 µs por valor; con el texto fijo y la copia al buffer (`tx_string`), ≈ 280 µs por línea, una vez cada 2 ms.

D. *Ventajas y límites*

+ Se lee directo en una terminal o en el Serial Plotter, y el fin de línea sincroniza solo.
+ En LabVIEW basta *VISA Read* con el carácter de fin `\n` y *Scan From String*; es el formato que sugiere el enunciado.
+ El PIC empieza a mandar al encenderse: el PC no tiene que pedir nada.
+ Sin nombres la línea es corta (11 bytes): a 500 líneas/s usa menos de la mitad de la UART.
– El PC depende del orden: el primer número es I y el segundo Q.
– Al PC llega 1 de cada 2 pares ($f_s = 500$ Hz); suficiente para señales $\lesssim 20$ Hz.
– Sin verificación de errores ni número de secuencia: una línea dañada se descarta si deja de tener la forma «número,número»; una perdida no se ve (se revisa el contador `adcLineasPerdidas` del PIC).
– El PC no puede mandar comandos (el control del PWM queda por definir).

V. DIAGRAMAS DE TIEMPO

Las Figs. 1–5 se generaron con el script `generar_diagramas_tiempo.m` (MATLAB). Los tiempos del ADC y de la UART son exactos: salen del datasheet (conversión de 28,5 µs, latencia de interrupción) o de los registros (periodo de 1 ms, 114 286 baud, línea de 962,5 µs). Los tiempos de CPU son una *previsión*: se estimaron contando instrucciones a 4 instrucciones/µs y se medirán con un pin marcador y el osciloscopio. Todas las figuras suponen una adquisición de 10 µs.

Figura 1. Dos periodos de muestreo (1 kHz) y una línea ASCII cada 2 pares

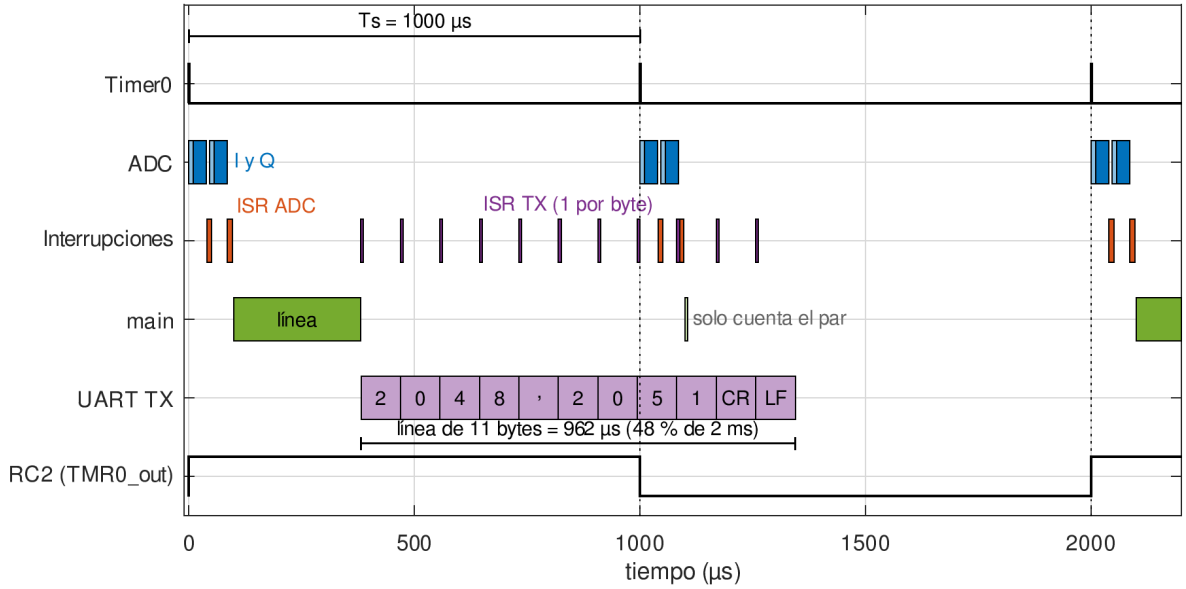


Fig. 1. Dos periodos de muestreo. El Timer0 dispara la conversión de I por hardware en $t = 0$ y en $t = 1\,000\,\mu s$; cada ráfaga $I + Q$ termina hacia los $97\,\mu s$. En el primer tick el programa principal arma la línea ($\approx 280\,\mu s$); en el segundo solo cuenta el par. La línea ocupa la UART $962{,}5\,\mu s$ ($48\,\%$ de $2\,ms$) y sus interrupciones de TX caen sobre la ráfaga siguiente, donde pueden retrasar unos $6\,\mu s$ el arranque de Q. RC2 saca la salida del Timer0: cambia de nivel en cada disparo.

Figura 2. Adquisición, conversión e interrupciones de I y Q

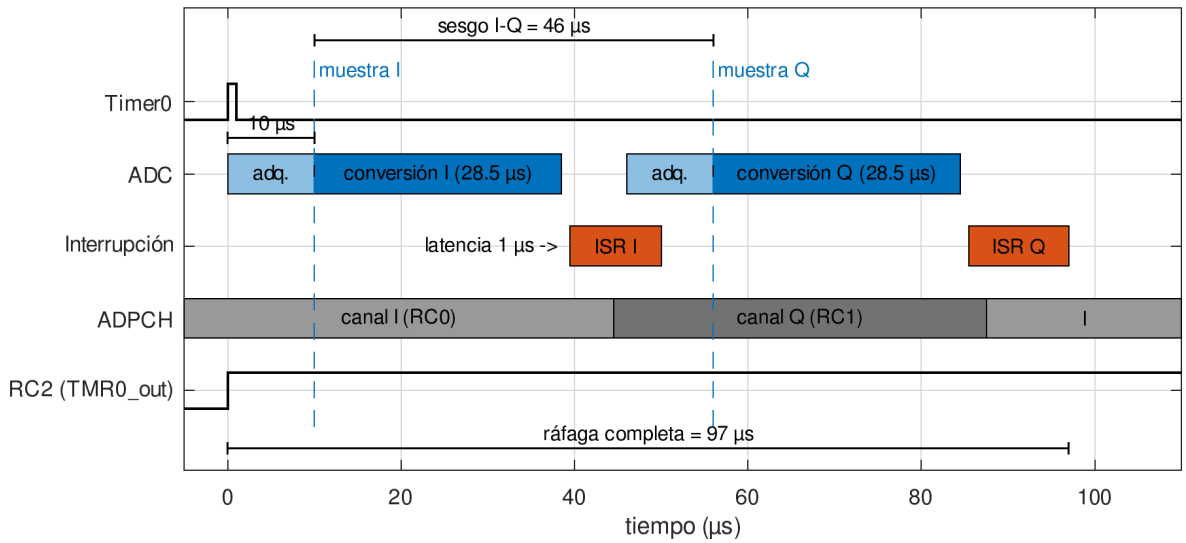


Fig. 2. Lectura del ADC. Cada canal tiene $10\,\mu s$ de adquisición y $28{,}5\,\mu s$ de conversión ($14\,T_{AD} + 2\,T_{CY}$). Al terminar I, la interrupción (latencia de $0{,}75\,\mu s$ to $1\,\mu s$) lee `ADRES`, cambia `ADPCH` a RC1 y arranca Q. Por eso la muestra de Q queda $\approx 46\,\mu s$ después de la de I.

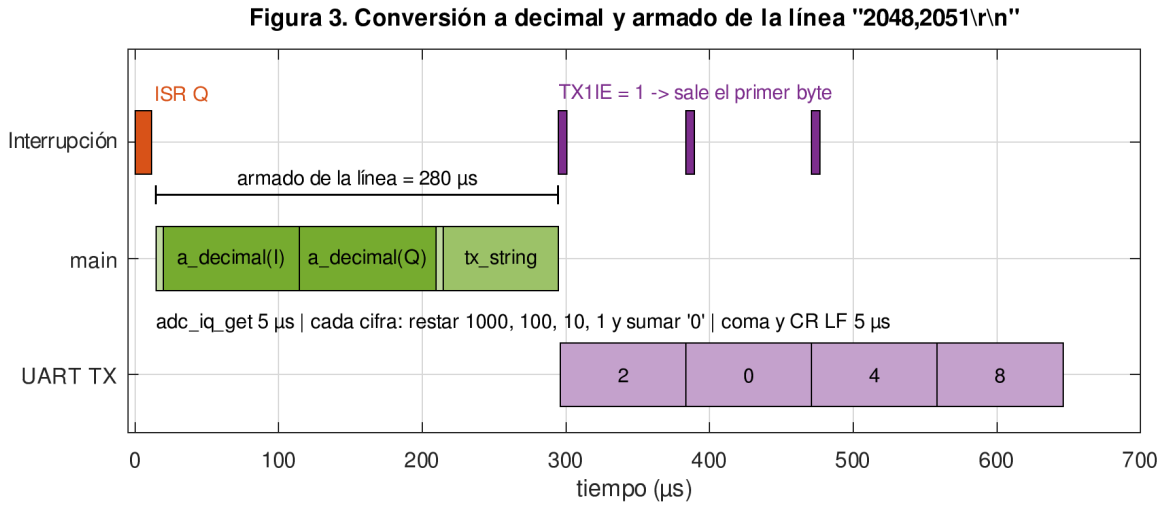


Fig. 3. Conversión a decimal y armado de la línea. Lo que más tarda es `a_decimal()` ($\approx$95 µs por valor), seguido de la copia de los 11 bytes al buffer de TX con `tx_string` ($\approx$80 µs). Al poner `TX1IE = 1`, la interrupción de TX carga los primeros bytes y la línea empieza a salir.

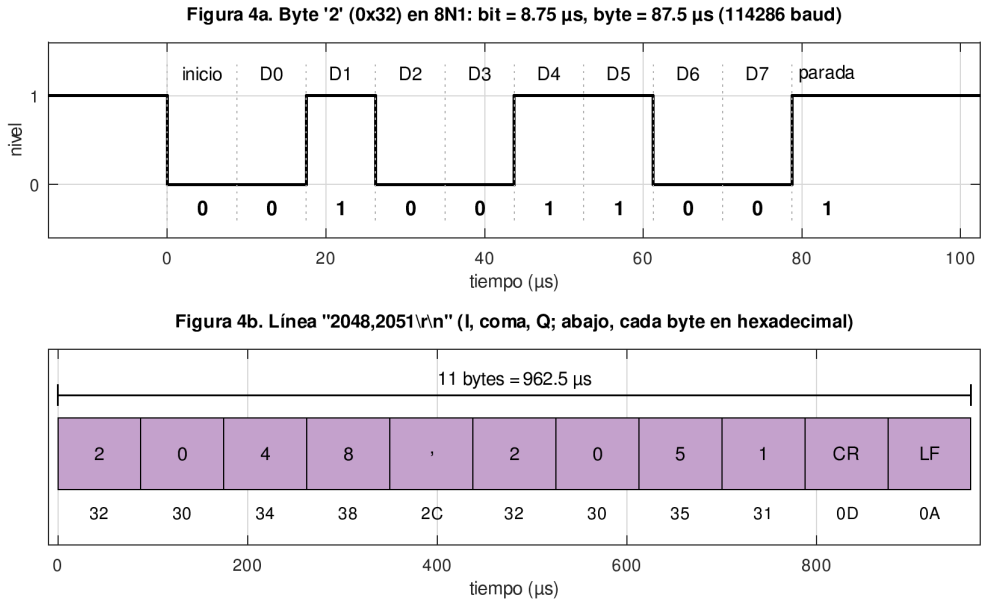


Fig. 4. (a) Byte `'2'` (`0x32`) en la línea: bit de inicio en 0, los 8 datos empezando por el menos significativo (0,1,0,0,1,1,0,0) y bit de parada en 1. Cada bit dura 8,75 µs y el byte 87,5 µs. (b) Línea completa del ejemplo $I = 2048$, $Q = 2051$: 11 bytes, 962,5 µs.

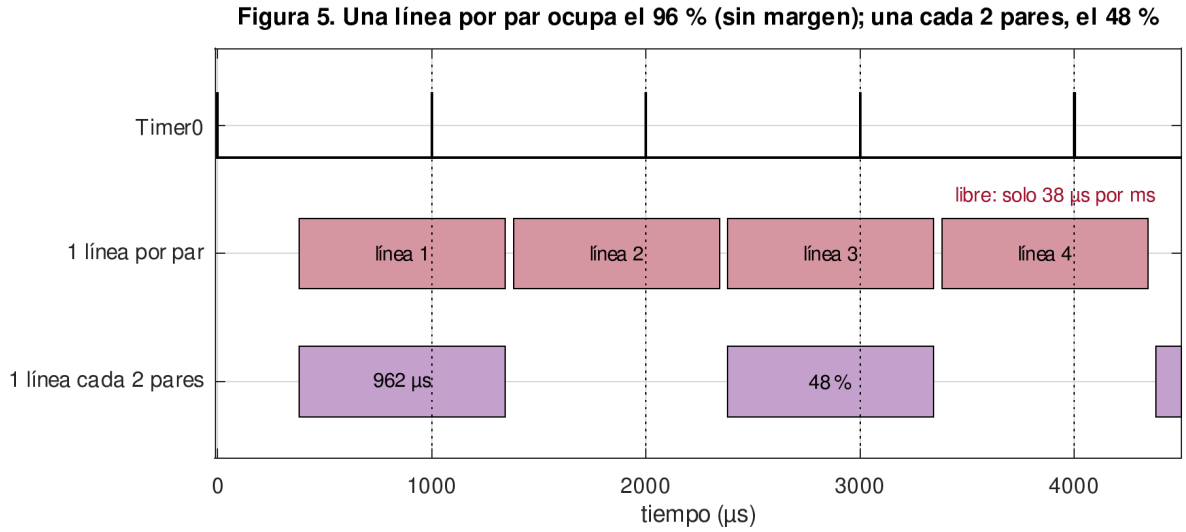


Fig. 5. Por qué una línea cada dos pares. Con una línea por par, cada línea ($962{,}5\,\mu s$) ocupa casi todo el periodo de $1\,ms$: la UART queda ocupada el $96\,\%$ y cualquier atraso llena el buffer de TX. Con una cada dos pares la línea queda libre el $52\,\%$ del tiempo.

Referencias

[1] Microchip Technology, *PIC16(L)F18426/46 Data Sheet*, DS40001985C (ADCC, EUSART, Timer0, parámetros AD20 y AD22).

[2] Texas Instruments, *SN74LVC1T45 Single-Bit Dual-Supply Bus Transceiver*, SCES515.

[3] Silicon Labs, *CP2102 Single-Chip USB to UART Bridge*.